FINAL YEAR PROJECT

PyTutor 3.0

Misconception-Aware AI Tutor for Novice Python Learners

Student Name

Supervisor Name

Academic presentation · September 2026

Outline

01	**Introduction**	Why novice Python learning needs more than correctness feedback
02	**Literature Review**	Existing methods, limitations, and the research gap
03	**Proposed Method / System Architecture**	Multi-stage decisions, diagnosis, tutoring, and feedback
04	**Experimental Results**	Dataset, model results, and experiment design
05	**Conclusion**	Contributions, limitations, and a compact demo route

Why novice Python learners need a dedicated tutor

Traditional grading

Correctness signal only

Wrong Answer 1/4 passed

No explanation of the underlying concept

General-purpose LLM

Rich explanations

But may reveal complete code

Copying can replace learning

Classroom reality

Teachers need class-level evidence

Which misconceptions recur?

Who needs targeted support?

Research motivation

Feedback should explain why the code fails, guide the next attempt, and preserve the learner’s agency.

Research gap

Existing strengths

- Automated assessment scales to many submissions
- LLMs can explain code in natural language
- Intelligent tutoring models learner state

But the pieces remain disconnected

No novice-focused integration of

interpretable misconception diagnosis
+ scaffolded hints
+ long-term learner tracking

Research question: Can a Python tutoring system combine diagnosis, pedagogical control, and learner tracking in one verifiable workflow?

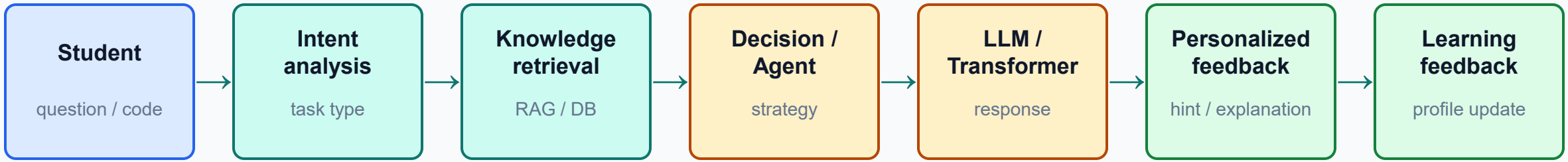
Existing methods and their limitations

Research direction	What it achieves	Limitation	PyTutor response
Automated grading	Fast correctness checks	Feedback remains shallow	M1–M8 misconception labels
LLM in CS education	Natural-language help	Answer leakage / instability	Progressive hints + validation
Intelligent tutoring	Structured pedagogy	Rules can be hard to scale	Regex → AST → LLM
Learner modelling	Tracks learning state	Often data- and model-heavy	Lightweight profile + decay

Synthesis

The opportunity is not another chatbot; it is a constrained tutoring pipeline with evidence at each decision point.

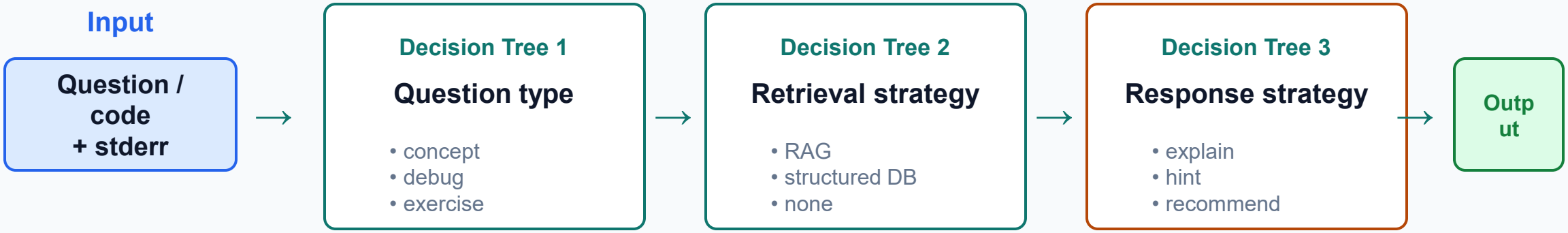
From student input to learning feedback



Why this structure?

- Separate retrieval from response generation
- Make pedagogical decisions explicit
- Turn each interaction into learner evidence

Multi-stage decision making



Decision trace: selected branch → retrieved evidence → pedagogical strategy → response constraint

Transformer role in the tutoring pipeline



System role

Semantic understanding · Intent recognition · Context modelling · Answer generation

The Transformer is one component inside a pedagogical control loop, not the whole tutor.

Three-layer misconception diagnosis

01 Regex rules

Known patterns
< 1 ms
no network

→

02 AST analysis

Structure + line evidence
~ ms
explainable

→

03 LLM fallback

Unseen variants
~ 500 ms
only when needed

```
Output: { misconception_id, confidence, evidence, ast_features,
strategy }
```

Fast → structured → broad

Progressive hinting and learner state

Pedagogical control

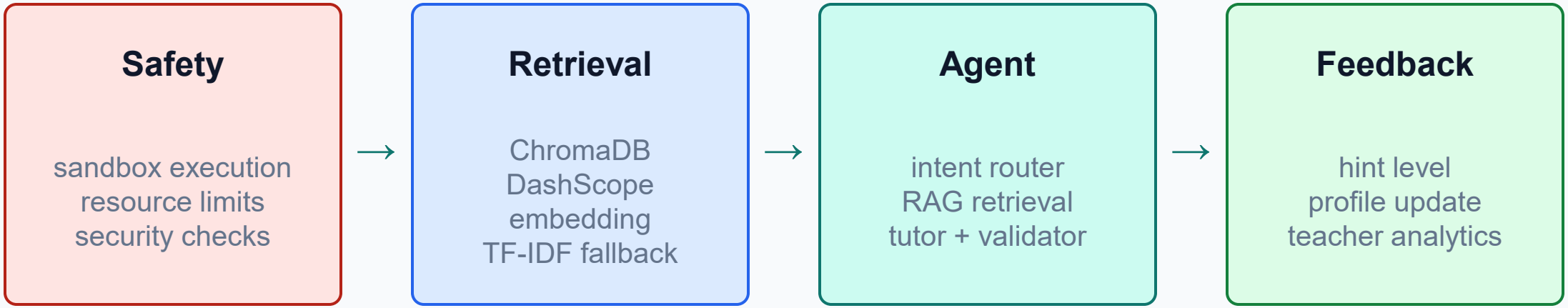


Learner profile

- weak_topics
- recent_misconceptions
- hint_dependency
- completed_lessons
- time decay

Policy: first attempt receives guidance, not a copied solution.

Agent orchestration, retrieval, and safety



Feature flags isolate algorithm updates so each mechanism can be tested, compared, and rolled back independently.

Experimental setup

Dataset

47 misconception cases
22 clean-code cases
10-case teaching benchmark
M1–M8 label coverage

Metrics

Accuracy
Precision / Recall
Macro F1
False-positive rate
Response quality
RAG: Hit@K / Recall@K / MRR

Evaluation scripts

run_v2_eval.py
run_rag_eval.py
run_judge_eval.py
Offline and API-dependent paths separated

Evidence boundary

AST-only results are local, offline measurements. Multi-Judge has no human reference; RAG is a 30-query single-author set.

Model results: local AST diagnosis

47 / 47
exact misconception matches

1.000
macro F1 across M1–M8

0%
false positives on 22 clean cases

< 1.2 ms
single diagnosis latency

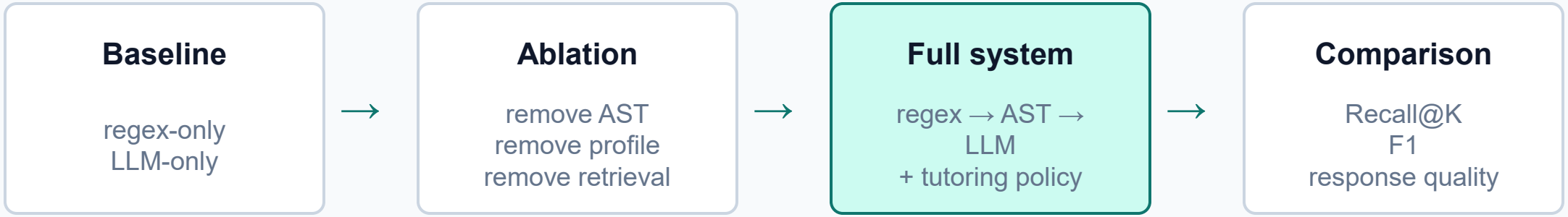
What the result supports

- Local structural analysis is fast and deterministic
- M1–M8 labels can become an intermediate representation
- Line-level evidence supports explanation and auditing

Interpretation

Strong evidence for the AST-only component.
Not a claim that the full tutoring system is solved.

Experiment design and measured results



Measured so far

AST 47/47 same-set · holdout 25/30 · 0 false positives RAG Hit@5 90.0% → 93.3% · Recall@5 40.7% → 50.1% · MRR 0.900 → 0.890

Faithfulness C1/C2: 0.683 / 0.647 (run 1), 0.667 / 0.650 (run 2) — C2 not better; 95% CI includes 0

Planned: expand labelled data, calibrate confidence, and compare against general-purpose coding assistants.

Interpretable confidence and its boundary

$$\text{logit} = \text{logit}(\text{PRIOR}) + \Sigma\ w_i \cdot \text{evidence}_i$$

```
confidence = clip(sigmoid(logit), 0.15, 0.95)
```

Evidence changes the score; the score is not a calibrated probability.

What is valid now

- Rank competing diagnoses
- Gate weak evidence
- Expose why a label was selected

What remains

- ≥ 200 human-labelled examples
- Fit weights with logistic regression
- Reliability diagram for calibration

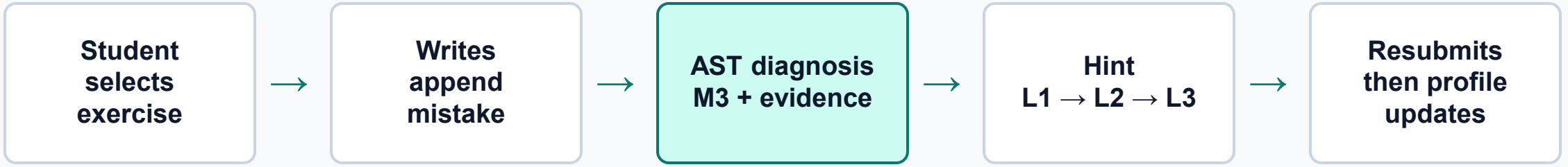
Boundary: weights are still heuristic; no Brier score or ECE is reported until at least 200 independently labelled examples exist.

Contributions

01	**Interpretable diagnosis**	Regex → AST → LLM with evidence and confidence
02	**Pedagogical control**	Seven strategies and five hint levels
03	**Learner adaptation**	Profile, time decay, and misconception history
04	**Verifiable engineering**	Offline evaluation, feature flags, and safety boundaries

Positioning: a runnable, evaluable, and explainable AI tutoring workflow — not merely an LLM interface.

Compact demo route



Live demonstration goal

Show the Student moving through the same research pipeline: input → diagnosis → guided response → learning feedback.

Negative branches: no-misconception code → no label (not a false positive); sandbox unavailable → not judged, no reward.

Next: confidence calibration · larger baselines · end-to-end stability

Thank you

PyTutor 3.0

Misconception-aware · scaffolded · profile-driven · verifiable